1. The slow query and its plan

This report is about one query in the order-tracking service of a fictional online shop. The support console has a screen that lists the paid orders of a single customer, newest first, and that screen had become noticeably slow as the orders collection grew. Staff complained that opening a customer's page left them waiting, and the complaint was serious enough that I was asked to find out why and to fix it.

The query behind the screen is the following. It filters on two equality conditions, the order status and the customer, and sorts the result by the date the order was placed, newest first, returning the first twenty documents.

db.orders.find({ status: "paid", customerId: "c-48213" }).sort({ createdAt: -1 }).limit(20)

Before adding anything, I asked the database to explain the query. The winning plan reported by explain() was the following, shown here as the stages from the outside in.

LIMIT, then SORT on createdAt descending, then COLLSCAN on the orders collection with the filter status equals paid and customerId equals c-48213.

Reading the plan from the inside out: the database starts with a collection scan, which means it opens every document in the collection and tests the filter against it. The matching documents are then handed to an in-memory sort stage, which orders them by createdAt, and only after that does the limit stage keep twenty. The collection scan is the expensive step, because its work grows with the number of orders in the whole collection rather than with the number of orders the customer actually has. The sort stage is a second cost, because it has to hold all matching documents before it can return any.

The only index on the collection at that time was the default index on the identifier field, which is of no use to a query that does not filter on it. That is why the planner had nothing better than a scan. The rest of this report describes the data, the workload the query belongs to, how I measured, the index I created, how I checked that the planner uses it, the alternatives I considered, and finally the reasoning behind the exact shape of the index.

2. The data

The orders collection belongs to a synthetic data set that I generated for this exercise, so that none of the numbers or identifiers describe a real business. It holds documents with a fairly stable shape. The fields that matter for this report are listed here with a short description of each, followed by the rest for completeness.

Field _id: the unique identifier of the order, generated by the database.

Field customerId: the identifier of the customer who placed the order, a string of 24 hexadecimal characters.

Field status: the stage of the order, one of pending, paid, packed, shipped, delivered or cancelled.

Field region: a short code for the sales region the order belongs to.

Field createdAt: the date and time the order was placed, stored as a native date.

Field updatedAt: the date and time of the latest change to the order.

Field total: the order total in the smallest currency unit, stored as an integer.

Field currency: a three-letter currency code.

Field sku: the code of the main product on the order, kept at the top level for convenience.

Field items: an array of line items, each with a product code, a quantity and a unit price.

Field shippingAddress: an embedded document with the delivery address.

Field billingAddress: an embedded document with the billing address, often equal to the shipping address.

Field channel: where the order came from, such as web, mobile or phone.

Field couponCode: an optional code the customer entered at checkout.

Field notes: optional free text left by the customer.

Field carrier: the delivery company, filled in once the order is packed.

Field trackingNumber: the carrier's tracking number, filled in once the order is shipped.

Field flags: an array of short labels that staff attach to orders for follow-up.

The documents are mostly small, a few kilobytes each, because most orders have between one and four line items. A handful of bulk orders have more than a hundred line items and are several times larger. I did not treat those as special in this exercise.

Customers vary a great deal in how many orders they have. Most have fewer than ten. A small number of business customers have thousands. The query in section 1 is run against all kinds of customers, but the support complaints came mainly from staff looking at the larger accounts, where the sort stage had many documents to hold.

The status field has six values, and the distribution is heavily uneven: most orders are delivered, a moderate share are paid or packed or shipped, and few are pending or cancelled. Paid orders are the group the support console looks at first, since those are the ones waiting for action. I mention this because the selectivity of a condition decides how useful an index on it would be on its own, which comes up again in section 7.

3. The workload

A query is never alone. Before choosing an index I wanted to know which other queries run against the same collection, so that I would not tune for one screen and accidentally leave the rest as they were. I took the query log of the sample week, grouped queries by their shape, meaning the set of fields filtered on, the sort and the limit, and counted each group. Forty shapes accounted for nearly all the traffic. They are listed here.

Shape 01: filter on status and customerId and createdAt, createdAt ascending, limit 20. This shape appeared 8219 times in the sample week, mostly from the export job.

Shape 02: filter on sku and status, total descending, limit 50. This shape appeared 7138 times in the sample week, mostly from the support console.

Shape 03: filter on status and region, no sort, limit 100. This shape appeared 6057 times in the sample week, mostly from the mobile app.

Shape 04: filter on region and createdAt, createdAt descending, limit 10. This shape appeared 4976 times in the sample week, mostly from the dashboard.

Shape 05: filter on customerId and status, createdAt ascending, limit 20. This shape appeared 3895 times in the sample week, mostly from the export job.

Shape 06: filter on customerId and createdAt, total descending, limit 50. This shape appeared 2814 times in the sample week, mostly from the support console.

Shape 07: filter on status and createdAt, no sort, limit 100. This shape appeared 1733 times in the sample week, mostly from the mobile app.

Shape 08: filter on status and customerId, createdAt descending, limit 10. This shape appeared 652 times in the sample week, mostly from the dashboard.

Shape 09: filter on status and customerId and createdAt, createdAt ascending, limit 20. This shape appeared 8571 times in the sample week, mostly from the export job.

Shape 10: filter on sku and status, total descending, limit 50. This shape appeared 7490 times in the sample week, mostly from the support console.

Shape 11: filter on status and region, no sort, limit 100. This shape appeared 6409 times in the sample week, mostly from the mobile app.

Shape 12: filter on region and createdAt, createdAt descending, limit 10. This shape appeared 5328 times in the sample week, mostly from the dashboard.

Shape 13: filter on customerId and status, createdAt ascending, limit 20. This shape appeared 4247 times in the sample week, mostly from the export job.

Shape 14: filter on customerId and createdAt, total descending, limit 50. This shape appeared 3166 times in the sample week, mostly from the support console.

Shape 15: filter on status and createdAt, no sort, limit 100. This shape appeared 2085 times in the sample week, mostly from the mobile app.

Shape 16: filter on status and customerId, createdAt descending, limit 10. This shape appeared 1004 times in the sample week, mostly from the dashboard.

Shape 17: filter on status and customerId and createdAt, createdAt ascending, limit 20. This shape appeared 8923 times in the sample week, mostly from the export job.

Shape 18: filter on sku and status, total descending, limit 50. This shape appeared 7842 times in the sample week, mostly from the support console.

Shape 19: filter on status and region, no sort, limit 100. This shape appeared 6761 times in the sample week, mostly from the mobile app.

Shape 20: filter on region and createdAt, createdAt descending, limit 10. This shape appeared 5680 times in the sample week, mostly from the dashboard.

Shape 21: filter on customerId and status, createdAt ascending, limit 20. This shape appeared 4599 times in the sample week, mostly from the export job.

Shape 22: filter on customerId and createdAt, total descending, limit 50. This shape appeared 3518 times in the sample week, mostly from the support console.

Shape 23: filter on status and createdAt, no sort, limit 100. This shape appeared 2437 times in the sample week, mostly from the mobile app.

Shape 24: filter on status and customerId, createdAt descending, limit 10. This shape appeared 1356 times in the sample week, mostly from the dashboard.

Shape 25: filter on status and customerId and createdAt, createdAt ascending, limit 20. This shape appeared 9275 times in the sample week, mostly from the export job.

Shape 26: filter on sku and status, total descending, limit 50. This shape appeared 8194 times in the sample week, mostly from the support console.

Shape 27: filter on status and region, no sort, limit 100. This shape appeared 7113 times in the sample week, mostly from the mobile app.

Shape 28: filter on region and createdAt, createdAt descending, limit 10. This shape appeared 6032 times in the sample week, mostly from the dashboard.

Shape 29: filter on customerId and status, createdAt ascending, limit 20. This shape appeared 4951 times in the sample week, mostly from the export job.

Shape 30: filter on customerId and createdAt, total descending, limit 50. This shape appeared 3870 times in the sample week, mostly from the support console.

Shape 31: filter on status and createdAt, no sort, limit 100. This shape appeared 2789 times in the sample week, mostly from the mobile app.

Shape 32: filter on status and customerId, createdAt descending, limit 10. This shape appeared 1708 times in the sample week, mostly from the dashboard.

Shape 33: filter on status and customerId and createdAt, createdAt ascending, limit 20. This shape appeared 627 times in the sample week, mostly from the export job.

Shape 34: filter on sku and status, total descending, limit 50. This shape appeared 8546 times in the sample week, mostly from the support console.

Shape 35: filter on status and region, no sort, limit 100. This shape appeared 7465 times in the sample week, mostly from the mobile app.

Shape 36: filter on region and createdAt, createdAt descending, limit 10. This shape appeared 6384 times in the sample week, mostly from the dashboard.

Shape 37: filter on customerId and status, createdAt ascending, limit 20. This shape appeared 5303 times in the sample week, mostly from the export job.

Shape 38: filter on customerId and createdAt, total descending, limit 50. This shape appeared 4222 times in the sample week, mostly from the support console.

Shape 39: filter on status and createdAt, no sort, limit 100. This shape appeared 3141 times in the sample week, mostly from the mobile app.

Shape 40: filter on status and customerId, createdAt descending, limit 10. This shape appeared 2060 times in the sample week, mostly from the dashboard.

Two observations come out of the list. First, the filter on status and customerId, with a sort on createdAt, is among the most frequent shapes, so the fix for it benefits many requests and not only the support screen. Second, several shapes filter on the same two or three fields in different combinations, which suggested from the start that a single well-chosen compound index might cover more than one of them.

4. Method and environment

All measurements were taken on a single development machine with the database running locally in a container, with the synthetic collection loaded once and left untouched between experiments. The machine had no other heavy workload during the runs, and I closed the editor and the browser tabs that I knew to be busy.

I used the database's built-in explain facility for plans, and the shell's own timing for the duration of a query. A query was run several times before I noted anything, so that the first, slower run that fills the cache was not mistaken for typical behaviour. I describe the plan in words in this report, because the stage names are what matter for the argument, and I kept the raw explain output in a text file next to my notes.

The tools I used were the database shell, a small script that picked a customer at random from the heavier accounts, and a spreadsheet for keeping a log of what I tried. The script exists because a query against one customer who happens to have very few orders would have hidden the problem, and one against a very large account would have exaggerated it. Picking from the heavier accounts matches the case the staff complained about.

I kept notes in the order I did things. The notes are the source of the sections that follow, and where a figure appears in this report it is copied from those notes and not recalculated. The routine entries from the working log are reproduced here so that a reader can see how the sessions were organised.

Log entry 01: confirmed that the filter matched some documents, then checked that the container had not been restarted, and moved on to the next step of the checklist for session 1.

Log entry 02: saved the raw explain output to the text file, then reran the query three times and kept the last run, and moved on to the next step of the checklist for session 1.

Log entry 03: cleared the shell history so that the next run started clean, then saved the raw explain output to the text file, and moved on to the next step of the checklist for session 1.

Log entry 04: checked that the container had not been restarted, then compared the stage names with the previous entry, and moved on to the next step of the checklist for session 1.

Log entry 05: compared the stage names with the previous entry, then picked a customer from the heavier accounts, and moved on to the next step of the checklist for session 1.

Log entry 06: noted the customer identifier in the spreadsheet, then cleared the shell history so that the next run started clean, and moved on to the next step of the checklist for session 2.

Log entry 07: reran the query three times and kept the last run, then noted the customer identifier in the spreadsheet, and moved on to the next step of the checklist for session 2.

Log entry 08: picked a customer from the heavier accounts, then confirmed that the filter matched some documents, and moved on to the next step of the checklist for session 2.

Log entry 09: confirmed that the filter matched some documents, then checked that the container had not been restarted, and moved on to the next step of the checklist for session 2.

Log entry 10: saved the raw explain output to the text file, then reran the query three times and kept the last run, and moved on to the next step of the checklist for session 2.

Log entry 11: cleared the shell history so that the next run started clean, then saved the raw explain output to the text file, and moved on to the next step of the checklist for session 2.

Log entry 12: checked that the container had not been restarted, then compared the stage names with the previous entry, and moved on to the next step of the checklist for session 3.

Log entry 13: compared the stage names with the previous entry, then picked a customer from the heavier accounts, and moved on to the next step of the checklist for session 3.

Log entry 14: noted the customer identifier in the spreadsheet, then cleared the shell history so that the next run started clean, and moved on to the next step of the checklist for session 3.

Log entry 15: reran the query three times and kept the last run, then noted the customer identifier in the spreadsheet, and moved on to the next step of the checklist for session 3.

Log entry 16: picked a customer from the heavier accounts, then confirmed that the filter matched some documents, and moved on to the next step of the checklist for session 3.

Log entry 17: confirmed that the filter matched some documents, then checked that the container had not been restarted, and moved on to the next step of the checklist for session 3.

Log entry 18: saved the raw explain output to the text file, then reran the query three times and kept the last run, and moved on to the next step of the checklist for session 4.

Log entry 19: cleared the shell history so that the next run started clean, then saved the raw explain output to the text file, and moved on to the next step of the checklist for session 4.

Log entry 20: checked that the container had not been restarted, then compared the stage names with the previous entry, and moved on to the next step of the checklist for session 4.

Log entry 21: compared the stage names with the previous entry, then picked a customer from the heavier accounts, and moved on to the next step of the checklist for session 4.

Log entry 22: noted the customer identifier in the spreadsheet, then cleared the shell history so that the next run started clean, and moved on to the next step of the checklist for session 4.

Log entry 23: reran the query three times and kept the last run, then noted the customer identifier in the spreadsheet, and moved on to the next step of the checklist for session 4.

Log entry 24: picked a customer from the heavier accounts, then confirmed that the filter matched some documents, and moved on to the next step of the checklist for session 5.

Log entry 25: confirmed that the filter matched some documents, then checked that the container had not been restarted, and moved on to the next step of the checklist for session 5.

Log entry 26: saved the raw explain output to the text file, then reran the query three times and kept the last run, and moved on to the next step of the checklist for session 5.

Log entry 27: cleared the shell history so that the next run started clean, then saved the raw explain output to the text file, and moved on to the next step of the checklist for session 5.

Log entry 28: checked that the container had not been restarted, then compared the stage names with the previous entry, and moved on to the next step of the checklist for session 5.

Log entry 29: compared the stage names with the previous entry, then picked a customer from the heavier accounts, and moved on to the next step of the checklist for session 5.

Log entry 30: noted the customer identifier in the spreadsheet, then cleared the shell history so that the next run started clean, and moved on to the next step of the checklist for session 6.

Log entry 31: reran the query three times and kept the last run, then noted the customer identifier in the spreadsheet, and moved on to the next step of the checklist for session 6.

Log entry 32: picked a customer from the heavier accounts, then confirmed that the filter matched some documents, and moved on to the next step of the checklist for session 6.

Log entry 33: confirmed that the filter matched some documents, then checked that the container had not been restarted, and moved on to the next step of the checklist for session 6.

Log entry 34: saved the raw explain output to the text file, then reran the query three times and kept the last run, and moved on to the next step of the checklist for session 6.

Log entry 35: cleared the shell history so that the next run started clean, then saved the raw explain output to the text file, and moved on to the next step of the checklist for session 6.

5. Creating the index and what happened

With the plan from section 1 and the workload from section 3 in hand, I created one compound index, naming the fields in this sequence: status, customerId, createdAt, with createdAt in descending direction.

db.orders.createIndex({ status: 1, customerId: 1, createdAt: -1 })

The database accepted the index and built it without errors on the synthetic collection. I then ran the same query from section 1 again. The query now returns in about 35 ms.

I also reran the other frequent shapes from section 3 that filter on status and customerId, to see whether they changed their plan. The ones that include both fields picked up the new index. Shapes that did not filter on status or customerId at all were unaffected, which is what one would expect, and I did not try to cover them with this index.

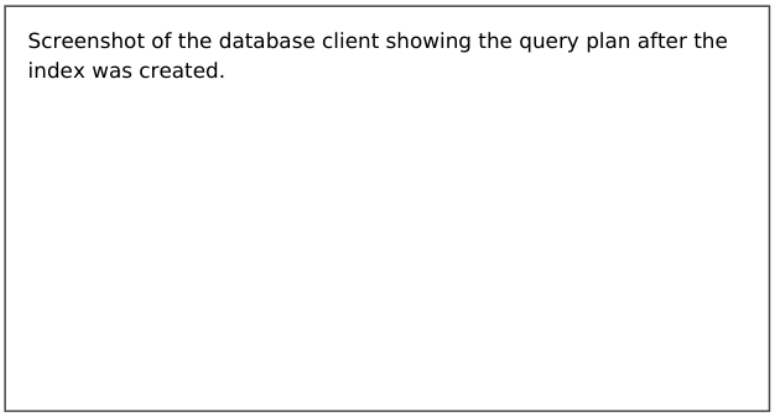
The next section shows how I confirmed that the faster result came from the new index and not from some other cause such as a warm cache.

6. Checking the new plan

A faster time on its own does not prove that an index is being used, because the cache may simply be warm. So I asked for the plan again. The stages are now different from section 1: a limit stage on top of a fetch stage on top of an index scan, and the index scan names the new compound index. There is no sort stage in the plan any more and there is no collection scan.

The explain output was longer than I wanted to reproduce in the report, so I took a screenshot of the plan from the database's graphical client. The screenshot below shows the plan after the index was created.

I also checked the number of index keys and documents that the plan reports having examined, and the two numbers are close to the number of documents returned, which is the pattern one hopes to see. The detail is in the raw output file.



7. Alternatives I considered

Before settling on the compound index I thought about several simpler options, and it is worth recording why I did not choose them.

A single-field index on status. The database can use only one such index for this query, and status has just six values, so the index narrows the search to a large fraction of the collection. The plan would still need to filter by customer and still sort in memory. It would help a little and not enough.

A single-field index on customerId. This narrows the search to one customer's orders, which is a big improvement over a collection scan. The remaining work is filtering by status and sorting by date, and for a business customer with thousands of orders the in-memory sort remains. It is a reasonable second choice and it is what I would try if the compound index were not possible.

Two separate single-field indexes. The planner can sometimes combine two indexes, but it still has to sort the combined result, so the sort stage stays.

An index on createdAt alone. The database could walk the index in date order and test each document against the filter, stopping after twenty matches. For a customer with many recent paid orders that is fast, and for a customer with few orders it degrades into walking nearly the whole index. It is unpredictable, which is the opposite of what the support staff need.

A partial index restricted to paid orders. This is attractive because paid orders are a minority of the collection. I decided against it because other frequent shapes in section 3 filter on other statuses, and I preferred a general solution to a narrow one for this exercise.

A text or search index. Nothing in the query is a text search, so this was never a serious candidate.

After looking at these options I chose the compound index in section 5. The reasoning for the order of its fields is the last piece, and the next section gives it.

8. Why the compound index has the fields in this order

The index is on status, then customerId, then createdAt descending. The order of the fields in a compound index is not cosmetic. It decides which queries the index can serve and how much work the database does for each.

The rule I followed has three parts: equality fields come first, the sort field comes next, and range fields come last. Both status and customerId are tested with equality in our query, so they go at the front. The sort field createdAt goes after them. Because the equality fields are fixed to a single value each, the entries in the index for that exact pair of values are already stored in createdAt order, and the database can read them in the order the query asks for and stop after twenty. That is why the sort stage disappeared from the plan in section 6.

If createdAt were first, the entries would be in date order for the whole collection, and the database would have to walk through entries of every customer and status to find the matching ones, which is the unpredictable behaviour described for the createdAt-only index in section 7. If customerId were first and status second, the index would work equally well for this query, because both are equality conditions. I put status first because several other frequent shapes in section 3 filter on status alone or on status and region, and a leading status field lets the index serve some of those as well, since a compound index can serve queries on a prefix of its fields.

Putting a range field before the sort field would have broken the property that makes the sort free, because within a range of values the entries are no longer in sort order. Our query has no range condition, so that does not arise here, but it is the reason the rule puts range fields last.

The direction of createdAt, descending, matches the sort in the query, which lets the database read the index forwards. An ascending index could be read backwards for the same effect, so the direction matters less than the position of the field, but I kept it equal to the query for clarity.

In short: equality fields first, then the sort field, then any range field. The index I created follows that order, and the plan in section 6 shows that the database uses it in the way I intended.